Certified PeTTa: Logic Programming Semantics, Prolog Compilation, and OSLF Integration in Lean 4

Zar Oruži*

July 18, 2026

Abstract

We present a Lean 4 formalization of the PeTTa language—the Prolog-based implementation of MeTTa—comprising 17,114 lines across 49 files with zero sorries and zero custom axioms. The formalization establishes a certified semantic stack from LP foundations through Prolog-style goal evaluation to an OSLF type system with Galois connection and Rust code export.

The main contributions are: (1) a full LP kernel with T_P, least Herbrand model via Tarski's theorem, SLD resolution, and assumption-free unification completeness; (2) correctness of MeTTaIL pattern matching on the `isMatchCorrect` fragment; (3) the MeTTaIL–LP bridge proving that supplied authored-rule applications belong to the compiled least model; (4) `PeTTaEval`, a pure evaluation relation with LP soundness for rule application; (5) `PeTTaCmd`, a stateful effects layer for `add-atom`, `remove-atom`, `get-atoms`, `progn`, and `prog1`; (6) a certified propositional connection-tableau chainer with DFS refinement and witness-level completeness; (7) a Prolog-style goal language (`PrologGoal`) with 14 goal forms including conjunctive/disjunctive goals, cut, negation, if-then-else, unification, and `findall`, together with an ISO conformance harness and a compilation pass (`compileExpr`) that translates PeTTa expressions into Prolog goals with correctness theorems; (8) a 4-argument MeTTa evaluation judgment (`MeTTaEval`) with grounded oracles, standard library derived forms, and typed evaluation; and (9) an OSLF pipeline instance composing `pettaSpaceToLangDef` with the generic OSLF type synthesis to produce a type system with automatic $\Diamond \dashv \Box$ Galois connection and Rust `language!{}` macro export, plus a GSLT forward fiber for categorical integration.

* "Oruži" denotes AI-assisted collaboration using Claude Code (Anthropic), ChatGPT, and Codex (OpenAI). Lean 4 formalization at `https://github.com/mettapedia`.

Together these establish a certified end-to-end pipeline from PeTTa source semantics through LP model theory to a deployable OSLF type system.

Contents

1 Introduction

MeTTa (Meta Type Talk) is the native programming language of the OpenCog Hyperon AGI framework [**?**]. It combines higher-order term rewriting with probabilistic inference and a first-class atomspace (knowledge graph), making

it a technically demanding target for formal verification. PeTTa is the operational, Prolog-based implementation of MeTTa, implemented as a transpiler from MeTTa surface syntax to SWI-Prolog.

Our goal in this paper is to give the pure, rule-driven fragment of PeTTa a certified semantic foundation. Logic programs with definite clauses provide exactly the right vehicle: the least Herbrand model semantics [**?**, **?**] is compositional, fixed-point-theoretic, and mechanically tractable in Lean 4.

Contribution overview. We develop and verify the following stack in Lean 4 (all with zero sorries and zero custom axioms):

1. **LP kernel** (Sections 3–5). T_P operator, least Herbrand model via Tarski's theorem, SLD resolution, assumption-free unification completeness, and the MeTTaIL–LP bridge proving least-model membership for supplied authored-rule applications.

2. **MeTTaIL pattern matching** (Section 4). Correctness of the algorithmic matcher on the `isMatchCorrect` fragment.

3. **PeTTa evaluation and LP soundness** (Section 6). The `PeTTaEval` relation captures pure evaluation; LP soundness reflects every `ruleApp` step in the least Herbrand model.

4. **Stateful effects** (Section 9). `PeTTaCmd` models atomspace mutations with state monotonicity.

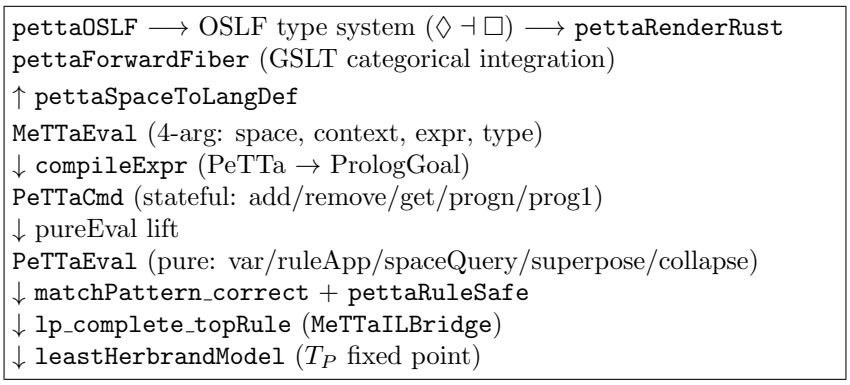
5. **Certified ATP/chainer** (Section 7). Connection-tableau DFS with soundness and witness-level completeness.

6. **Prolog goal language and compilation** (Section 11). A `PrologGoal` type with conjunction, disjunction, `findall`, and `assert`; a compilation pass `compileExpr` from PeTTa expressions to Prolog goals with correctness theorems.

7. **4-argument MeTTa evaluation** (Section 13). `MeTTaEval s ctx expr typ` with grounded oracles, standard library derived forms (`if-equal`, `car-atom`, `let`), and type-driven evaluation.

8. **OSLF pipeline and GSLT integration** (Section 14). `pettaOSLF` composes the PeTTa space into the generic OSLF type synthesis, obtaining an automatic $\Diamond \dashv \Box$ Galois connection. Rust `language!{}` macro export and a unit-indexed GSLT forward fiber complete the integration.

Architecture diagram. The certified pipeline:



Paper structure. Sections 2–5 cover background, the LP kernel, MeTTaIL pattern matching, and the MeTTaIL–LP bridge. Section 6 presents `PeTTaEval` and LP soundness. Section 7 adds the certified ATP/chainer layer. Section 9 covers the stateful effects layer. Section 10 formalises the MeTTa type system fragment. Section 11 presents the Prolog goal language and `compileExpr`. Section 13 covers the 4-argument `MeTTaEval` judgment, grounded oracles, and standard library forms. Section 14 integrates the OSLF pipeline and GSLT fiber. Section 15 discusses related work and Section 16 concludes.

2 Background

Logic programming. A *definite clause program* (or LP) is a finite set of *Horn clauses* $H \leftarrow B_1, \ldots, B_n$ where H and each B_i are atoms over a fixed signature. The *Herbrand base* is the set of all ground atoms constructible from the function and constant symbols in the signature. The immediate consequence operator T_P maps an interpretation I (a subset of the Herbrand base) to the set of all ground atoms that are immediate consequences of applying a grounded clause whose body is satisfied by I. Since T_P is monotone on the complete lattice $(\mathcal{P}(\mathrm{HB}), \subseteq)$, the Knaster–Tarski theorem guarantees a least fixed point $T_P \uparrow \omega$, which equals the least Herbrand model [**?**, **?**].

MeTTa and PeTTa. MeTTa expressions are terms over a *type-free* base (atoms, applications, variables). The semantics of a MeTTa program is given by its rewrite rules `(= lhs rhs)`: an expression reduces to all terms reachable by matching its subexpressions against LHS patterns and substituting the corresponding RHS. The non-determinism is explicit: `superpose` collects alternatives and `collapse` gathers all answers via `findall`-style comprehension.

PeTTa [**?**] is the Prolog-based implementation of MeTTa. The transpiler (`transpiler.pl`) converts MeTTa rules into Prolog clauses; `match &self` queries are handled by `match_term/3` in `spaces.pl`; `superpose`/`collapse` map to Prolog disjunction and `findall/3`.

Lean 4. Lean 4 [**?**] is a dependently-typed theorem prover and programming language. Our formalization uses Mathlib [**?**] for Tarski's fixed-point theorem (`OrderHom.lfp`) and basic order theory. The `@[simp]` attribute marks rewrite lemmas for the `simp` tactic; `decide` proves decidable propositions by kernel reduction.

3 The LP Kernel

The LP kernel lives in `Mettapedia/Logic/LP/` and spans three files: `Substitution.lean` (206 lines), `Semantics.lean` (204 lines), and `SLD.lean` (169 lines), totalling 579 lines.

3.1 Signature and Terms

```
structure LPSignature where
  vars            : Type
  constants       : Type
  functionSymbols : Type
  functionArity   : functionSymbols → Nat
  relationSymbols : Type
  relationArity   : relationSymbols → Nat

inductive Term (sigma : LPSignature) where
  | var   : sigma.vars → Term sigma
  | const : sigma.constants → Term sigma
  | app   : (f : sigma.functionSymbols) →
            (Fin (sigma.functionArity f) → Term sigma) → Term sigma

structure Atom (sigma : LPSignature) where
  symbol : sigma.relationSymbols
  args   : Fin (sigma.relationArity symbol) → Term sigma
```

Ground terms and atoms are defined analogously with `GroundTerm` and `GroundAtom`, using the same finitely-typed argument representation. A *grounding* `Grounding sigma := sigma.vars -> GroundTerm sigma` maps each LP variable to a ground term; applying it to a `Term` or `Atom` yields the corresponding `GroundTerm` or `GroundAtom`.

3.2 Clauses and Knowledge Bases

```
structure Clause (sigma : LPSignature) where
  head : Atom sigma
  body : List (Atom sigma)

structure GroundClause (sigma : LPSignature) where
  head : GroundAtom sigma
  body : List (GroundAtom sigma)

abbrev Program (sigma)       := List (Clause sigma)
abbrev Interpretation (sigma) := Set (GroundAtom sigma)
```

```
structure KnowledgeBase (sigma : LPSignature) where
  prog : Program sigma
  db   : Set (GroundAtom sigma)   -- extensional (EDB) ground atoms
```

The `db` field holds extensional (EDB) ground facts; `prog` holds intensional (IDB) definite clauses.

3.3 Least Herbrand Semantics

The immediate consequence operator:

```
noncomputable def T_P_LP {sigma : LPSignature}
    (kb : KnowledgeBase sigma) (I : Interpretation sigma) :
    Interpretation sigma :=
  kb.db
  union { a | ∃ (c : Clause sigma) (g : Grounding sigma),
              c in kb.prog
              /\ g.groundAtom c.head = a
              /\ ∀ b in c.body, g.groundAtom b in I }
```

Monotonicity of T_P:

Theorem 1 (Monotonicity)**.** *T_P is monotone: if $I \subseteq J$ then $T_P(I) \subseteq T_P(J)$.*

Proof. Straightforward: for each clause witness (c, g) satisfying the body in I, the same witness satisfies the body in $J \supseteq I$. □

Packaging T_P as an `OrderHom` (an order-homomorphism on the complete lattice of sets), Tarski's least-fixed-point theorem gives the least Herbrand model:

```
noncomputable def leastHerbrandModel {sigma : LPSignature}
    (kb : KnowledgeBase sigma) : Interpretation sigma :=
  OrderHom.lfp (T_P_LP_orderHom kb)
```

The following three theorems are the cornerstones of the LP semantics:

Theorem 2 (`leastHerbrandModel_fixpoint`)**.** $T_P(\mathrm{LHM}(kb)) = \mathrm{LHM}(kb)$.

Theorem 3 (`leastHerbrandModel_clause`)**.** *If $c \in kb.prog$, g is a grounding, and for all $b \in c.body$, $g(b) \in \mathrm{LHM}(kb)$, then $g(c.head) \in \mathrm{LHM}(kb)$.*

Theorem 4 (`leastHerbrandModel_least`)**.** *If $T_P(I) \subseteq I$, then $\mathrm{LHM}(kb) \subseteq I$.*

The proofs are one-liners using `OrderHom.isFixedPt_lfp` and `OrderHom.lfp_le` from Mathlib. Theorem 3 follows immediately from Theorem 2 and the definition of T_P: if the body holds in the model (which is a fixed point), then the head holds by one application of T_P.

3.4 SLD Resolution and All-Solutions SLD

The file `SLD.lean` (169 lines) formalises SLD resolution as an inductive type:

```
inductive SLDTree (sigma : LPSignature) (kb : KnowledgeBase sigma)
    : List (Atom sigma) → Subst sigma → Prop where
  | success : SLDTree sigma kb [] theta
  | resolve : ...
```

The soundness theorem (`sldTree_sound`) states that if a tree witnesses a successful derivation of a goal G with substitution θ, then the grounded goal $\theta(G)$ is in the least Herbrand model. The file `SLDAll.lean` (205 lines) provides `sldSearchAll`, which collects all answer substitutions, together with its soundness theorem.

3.5 Operational Completeness: Unification and SLD

The LP completeness layer extends the kernel with explicit executable-success theorems.

Assumption-free semantic completeness of unification. `Logic/LP/UnificationComplete.lean` (513 lines) internalizes the semantic-to-derivation bridge and proves:

Theorem 5 (`unifyFuel_exists_of_unifies`)**.** *If an equation list is semantically unifiable, then* ***`unifyFuel`*** *succeeds for some finite fuel:*

$$(\exists \delta,\ \textit{Unifies}\ \delta\ \textit{eqs}) \Rightarrow (\exists \textit{fuel}\ \theta,\ \textit{unifyFuel}\ \textit{fuel}\ \textit{eqs} = \textit{some}\ \theta).$$

Function-free and ground specializations are also proved:

- `unifyFuel_exists_of_unifies_functionFree`
- `unifyFuel_exists_of_unifies_ground`
- `unifyFuel_exists_of_unifies_functionFree_ground`

with canary theorems for occurs-check rejection and non-occurs success in `UnificationCompletenessCanaries.lean`.

SLD completeness bundles from least-model lifting. `Logic/LP/SLDCompute.lean` (485 lines) introduces `SLDWitness` and proves that witness existence implies executable success (`sldSearch_complete_bounded`, `sldQuery_complete_bounded`). The main endpoint is:

Theorem 6 (`sldQuery_complete_of_semantic_lift`)**.** *Given lifting data from least-model facts and one-step rule closures to* ***`SLDWitness`****, every least-Herbrand-model ground atom has a successful executable query:*

$$ga \in \mathrm{LHM}(kb) \Rightarrow \exists \textit{fuel}\ \theta,\ \textit{sldQuery}\ kb.prog\ ga\ \textit{fuel} = \textit{some}\ \theta.$$

The bundled endpoint `sldQuery_complete_sound_bundle_of_semantic_lift` adds soundness of returned substitutions in the same theorem. Canonical kit instances in `SLDCompletenessKit.lean` (261 lines) provide reusable hypothesis packages for concrete KB classes (e.g., nullary fact-only and nullary unary-body fragments).

4 MeTTaIL Pattern Matching Correctness

`OSLF/MeTTaIL/Match.lean` (201 lines) implements the algorithmic pattern matcher; `OSLF/MeTTaIL/MatchSpec.lean` (821 lines) proves its correctness.

4.1 MeTTaIL Syntax

MeTTaIL uses a *locally nameless* representation [**?**]: α-equivalent patterns are syntactically identical, so no renaming is needed during matching or substitution.

```
inductive Pattern where
  | fvar        : String → Pattern        -- free variable (
      metavariable)
  | bvar        : Nat → Pattern           -- de Bruijn bound
      variable
  | apply       : String → List Pattern → Pattern
  | lambda      : Pattern → Pattern
  | multiLambda : Nat → Pattern → Pattern
  | collection  : CollType → List Pattern → Option String → Pattern
  | subst       : Pattern → Pattern → Pattern    -- explicit
      substitution

inductive CollType where | vec | hashBag | hashSet

structure RewriteRule where
  left     : Pattern
  right    : Pattern
  premises : List Pattern    -- [] for unconditional rules
```

The `fvar` constructor represents *pattern metavariables*: they match any term and produce a variable binding. The `bvar n` constructor is a de Bruijn index for bound variables; it matches only `bvar m` with $m = n$. The `.subst body repl` form encodes explicit β-reduction: `applyBindings` evaluates it by calling `openBVar 0 repl body`.

4.2 Algorithmic Matching

```
def matchPattern (pat term : Pattern) : List Bindings :=
  match pat, term with
  | .fvar x, t                          => [[(x, t)]]
  | .bvar n, .bvar m                    => if n == m then [[]] else
      []
  | .apply c1 pargs, .apply c2 targs    =>
      if c1 == c2 && pargs.length == targs.length
```

```
      then matchArgs pargs targs
      else []
  | .lambda bodyPat, .lambda bodyConcrete =>
      matchPattern bodyPat bodyConcrete
  | .multiLambda npat bodyPat, .multiLambda nconc bodyConcrete =>
      if npat == nconc then matchPattern bodyPat bodyConcrete else
          []
  | .collection ct1 pelems rest1, .collection ct2 telems _rest2 =>
      if ct1 == ct2 then matchBag pelems rest1 ct1 telems else []
  | .subst pbody prepl, .subst tbody trepl =>
      (matchPattern pbody tbody).flatMap fun b1 =>
        (matchPattern prepl trepl).filterMap (mergeBindings b1)
  | _, _                                  => []
```

The function is mutually recursive with `matchArgs` (pairwise argument matching) and `matchBag` (multiset matching). Termination is by `sizeOf pat`. The return type `List Bindings` encodes non-determinism: bag matching may succeed in multiple ways.

`applyBindings` substitutes `fvar` metavariables by their bindings and evaluates `.subst` nodes:

```
def applyBindings (bindings : Bindings) (rhs : Pattern) : Pattern
    :=
  match rhs with
  | .fvar x   => match bindings.find? (fun p => p.1 == x) with
                 | some (_, val) => val
                 | none          => .fvar x
  | .bvar n   => .bvar n
  | .apply c args => .apply c (args.map (applyBindings bindings))
  | .lambda body  => .lambda (applyBindings bindings body)
  | .multiLambda n body => .multiLambda n (applyBindings bindings
      body)
  | .subst body repl =>
      openBVar 0 (applyBindings bindings repl)
                 (applyBindings bindings body)
  | .collection ct elems rest =>
      let elems' := elems.map (applyBindings bindings)
      let restElems := match rest with
        | some rv => match bindings.find? (fun p => p.1 == rv) with
                     | some (_, .collection _ relems _) => relems
                     | _ => []
        | none => []
      .collection ct (elems' ++ restElems) none
```

4.3 The `isMatchCorrect` Fragment

Not every pattern p satisfies $\texttt{applyBindings}(bs, p) = t$ for all $bs \in \texttt{matchPattern}(p, t)$. The two problematic constructors are:

- `.collection`: bag matching picks elements out of order, so the reconstituted pattern may have a different element permutation than t.
- `.subst`: `applyBindings` evaluates the substitution node via `openBVar`, which is not the identity.

The Boolean predicate `Pattern.isMatchCorrect` excludes both:

```
def Pattern.isMatchCorrect (p : Pattern) : Bool :=
  match p with
  | .fvar _           => true
  | .bvar _           => true
  | .apply _ args     => args.all isMatchCorrect
  | .lambda body      => isMatchCorrect body
  | .multiLambda _ b  => isMatchCorrect b
  | .subst _ _        => false
  | .collection _ _ _ => false
```

4.4 Correctness Theorem

The proof architecture in `MatchSpec.lean` is: (1) define relational specifications `MatchRel`, `MatchArgsRel`, `MatchBagRel` as mutual inductives independent of the algorithm; (2) prove algorithmic soundness (if $bs \in$ `matchPattern`(p, t) then `MatchRel`(p, t, bs)) by strong induction on `sizeOf pat`; (3) prove that `MatchRel` implies `applyBindings bs pat = t` on the `isMatchCorrect` fragment (by structural induction on `MatchRel`).

Theorem 7 (`matchPattern_correct`)**.** *If $bs \in$ `matchPattern`(pat, t) and `pat.isMatchCorrect` $=$ `true`, then `applyBindings`$(bs, pat) = t$.*

Proof. By the soundness bridge, $bs \in$ `matchPattern`(pat, t) implies `MatchRel`(pat, t, bs). Then `matchRel_correct` (structural induction on `MatchRel`, using the `isMatchCorrect` guard to exclude collection and subst cases) gives `applyBindings`$(bs, pat) = t$. □

The `isMatchCorrect` guard is sharp:

Theorem 8 (`matchPattern_correct_false`)**.** *There exist a pattern pat, a term t, and bindings bs such that $bs \in$ `matchPattern`(pat, t) yet `applyBindings`$(bs, pat) \neq t$.*

Proof. Counterexample:

$$pat = \mathrm{vec}(x, y), \quad t = \mathrm{vec}(b, a).$$

Bag matching with $i = 1$ first yields $bs = [(\text{"y"}, b), (\text{"x"}, a)]$, so

$$\texttt{applyBindings}(bs, pat) = [a, b] \neq [b, a] = t.$$

□

5 The MeTTaIL–LP Bridge

The bridge lives in `Logic/LP/MeTTaILBridge.lean` (847 lines).

5.1 LP Signature for MeTTaIL

```
abbrev mettailLPSig : LPSignature where
  vars            := String
  constants       := String
  functionSymbols := Unit      -- single binary pairing function
  functionArity   := fun _ => 2
  relationSymbols := Unit      -- single "reduces" relation
  relationArity   := fun _ => 2
```

Using `abbrev` (transparent) rather than `def` (opaque) is essential: it makes `mettailLPSig.functionArity () = 2` definitionally true, so that the `Matrix.cons` expression `![a, b]` typechecks for `Fin (mettailLPSig.functionArity ()) -> T`.

Patterns are encoded as binary pair-spine trees:

```
def pairTerm (a b : Term mettailLPSig) : Term mettailLPSig :=
  Term.app () ![a, b]

-- Encoding:
--   .fvar x     →  Term.var x               (LP variable)
--   .bvar n     →  Term.const "bv:n"        (ground token)
--   .apply f as →  pair("f_f", spine(as))
--   .lambda b   →  pair("lambda", enc(b))
--   etc.
```

`patternToLPTerm` is mutually recursive with `patternListToLPSpine` (right-spine encoding of lists). The *ground* version `patternToLPGroundTerm` treats `.fvar x` as `GroundTerm.const "fv:x"` (making the variable name part of the constant).

The *fragment* predicate `morkTranslatable` (inherited from the MORK bridge) excludes patterns with `.subst` and rest-variable `.collection` (i.e., `.collection ct elems (some rv)`):

```
-- from MORK bridge:
def morkTranslatable : Pattern → Bool
  | .fvar _              => true
  | .bvar _              => true
  | .apply _ args        => morkTranslatableList args
  | .lambda body         => morkTranslatable body
  | .multiLambda _ body  => morkTranslatable body
  | .collection _ _ none => morkTranslatableList elems
  | .collection _ _ _    => false   -- rest variable: excluded
  | .subst _ _           => false   -- explicit subst: excluded
```

5.2 Commutation Lemma

The key algebraic property is that grounding and encoding commute:

Lemma 9 (groundTerm_commutes_applyBindings)**.** *For* *morkTranslatable* *patterns* p *and bindings* bs*:*

$$\begin{aligned} &\mathit{bindingsToGrounding}(bs).\mathit{groundTerm}(\\ &\quad \mathit{patternToLPTerm}(p))\\ &= \mathit{patternToLPGroundTerm}(\\ &\quad \mathit{applyBindings}(bs, p)) \end{aligned}$$

Proof. By mutual well-founded recursion on `sizeOf p` and `sizeOf ps` (for the list version). The interesting cases:

- `.fvar x`: if bs contains (x, v), both sides evaluate to `patternToLPGroundTerm v`; otherwise both give `GroundTerm.const "fv:x"`.
- `.bvar n`: both sides give `GroundTerm.const "bv:n"` (const on both sides by construction).
- `.apply c args`: uses `groundTerm_pairTerm` (@[*simp*]) and the induction hypothesis for the spine.
- `.subst`, `.collection _ _ (some _)`: excluded by `morkTranslatable`.

The helper `@[simp]` lemma `groundTerm_pairTerm` establishes that grounding distributes over `pairTerm` via `Matrix.cons_val_zero` and `Matrix.cons_val_one`. □

5.3 Authored Rule-Application Encoding

Rewrite rules are compiled to LP clauses with empty bodies:

```
def rewriteRuleToLPClause (r : RewriteRule) : Clause mettailLPSig
    where
  head.symbol := ()
  head.args   := ![patternToLPTerm r.left, patternToLPTerm r.right]
  body        := []
```

Theorem 10 (ruleApplication_mem_leastHerbrandModel)**.** *Let* $r \in \mathit{lang.rewrites}$ *be a rule with* $r.\mathit{premises} = []$ *and* $\mathit{morkTranslatable}(r.left) = \mathit{true}$*,* $\mathit{morkTranslatable}(r.right) = \mathit{true}$*. If* $\mathit{applyBindings}(bs, r.left) = p$ *and* $\mathit{applyBindings}(bs, r.right) = q$*, then* $\mathit{encodeReduces}(p, q) \in \mathrm{LHM}(\mathit{languageDefToLPKB}(\mathit{lang}))$*.*

Proof. (1) The LP clause $c =$ `rewriteRuleToLPClause`(r) is in `KB.prog` (by `List.mem_append` and `List.mem_map`). (2) The body of c is empty, so the body condition is vacuously satisfied. (3) The commutation lemma (Lemma 9) gives:

$$g.\texttt{groundTerm}(\texttt{patternToLPTerm}(r.left)) = \texttt{patternToLPGroundTerm}(p)$$

where g = `bindingsToGrounding`(bs), and similarly for $r.right$ and q. (4) A manual finitary argument over `Fin 2` (using `Matrix.cons_val_zero` and `Matrix.cons_val_one`) gives g.`groundAtom`$(c.head)$ = `encodeReduces`(p, q). (5) Theorem 3 (`leastHerbrandModel_clause`) with steps (1)–(4) gives the conclusion; using Theorem 2 to convert from $T_P(LHM)$ to LHM. □

5.4 Authored Contextual Boundary

The LP compiler does not infer contextual closure from collection representation. Such closure would grant every language an accidental reduction policy. Instead, contextual authority is expressed by an authored `Premise.congruence` and interpreted by the least relation `ContextualStep.Step`. An LP backend for that relation must compile the actual authored contextual rule and prove agreement with it; this module deliberately introduces no generic collection-descent clauses.

The theorem above is therefore a forward encoding result for a supplied rule application, not a biconditional between arbitrary LP membership and language reduction. The generated clauses are not range-restricted—variables may occur in a head while the body is empty—so the least model records ground instances of authored rules and can over-approximate operational reachability.

The LP-translatability predicate is:

```
def lpTranslatable (r : RewriteRule) : Bool :=
  morkTranslatable r.left && morkTranslatable r.right &&
  Pattern.isMatchCorrect r.left
```

6 PeTTa Evaluation and LP Soundness

6.1 PeTTa Atomspace

`Languages/MeTTa/PeTTa/SpaceSemantics.lean` (160 lines) defines the atomspace structure:

```
structure PeTTaSpace where
  facts : List Pattern    -- EDB ground atoms
  rules : List RewriteRule

namespace PeTTaSpace
  def empty   : PeTTaSpace := { facts := [], rules := [] }
  def addAtom (s : PeTTaSpace) (p : Pattern) : PeTTaSpace :=
    { s with facts := p :: s.facts }
  def removeAtom (s : PeTTaSpace) (p : Pattern) : PeTTaSpace :=
    { s with facts := s.facts.filter (fun x => not (x == p)) }
  def addRule (s : PeTTaSpace) (r : RewriteRule) : PeTTaSpace :=
    { s with rules := r :: s.rules }
```

The space query operation models `(match &self pat tmpl)` in MeTTa:

```
def spaceMatch (s : PeTTaSpace) (pat tmpl : Pattern) : Answers :=
  s.facts.flatMap fun fact =>
```

```
    (matchPattern pat fact).map fun bs => applyBindings bs tmpl
```

Soundness and completeness of `spaceMatch` are proven: every answer arises from a fact-match pair, and every fact-match pair produces an answer.

6.2 PeTTa Evaluation Relation

`Languages/MeTTa/PeTTa/Eval.lean` (187 lines) defines the inductive evaluation judgment:

```
inductive PeTTaEval (s : PeTTaSpace) : Pattern → Answers → Prop
    where

  | var (x : String) :
      PeTTaEval s (.fvar x) [.fvar x]

  | bvar (n : Nat) :
      PeTTaEval s (.bvar n) [.bvar n]

  | ground (c : String) :
      PeTTaEval s (.apply c []) [.apply c []]

  | ruleApp (r : RewriteRule) (bs : Bindings) (p q : Pattern)
      (hr    : r in s.rules)
      (hprem : r.premises = [])
      (hm    : bs in matchPattern r.left p)
      (hq    : applyBindings bs r.right = q) :
      PeTTaEval s p [q]

  | spaceQuery (pat tmpl : Pattern) (results : Answers)
      (hres : results = s.spaceMatch pat tmpl) :
      PeTTaEval s (.apply "match" [.apply "&self" [], pat, tmpl])
          results

  | superpose (alts : List Pattern) :
      PeTTaEval s (.apply "superpose" [.collection .vec alts none])
           alts

  | collapse (p : Pattern) (answers : Answers)
      (h : PeTTaEval s p answers) :
      PeTTaEval s (.apply "collapse" [p]) [.collection .vec answers
           none]
```

The alignment with the HE MeTTa specification and the PeTTa transpiler is:

MeTTa spec	**PeTTaEval**	**PeTTa (Prolog)**
`metta(Variable, ...)`	`var`	clause head for fvar
`metta(Atom, ...)`	`ground`	fact clause
`metta_call(rhs, ...)`	`ruleApp`	top-level rule clause
`metta(match &self ...)`	`spaceQuery`	`match_term/3`
`superpose-bind`	`superpose`	Prolog `;` disjunction
`collapse-bind`	`collapse`	`findall/3`

Key properties proven in `Eval.lean`:

- `petta_eval_var_case`: `PeTTaEval s (.fvar x) [.fvar x]` holds for all x.
- `petta_eval_ruleApp_singleton`: rule application always produces exactly one answer.
- `petta_eval_superpose_nil`: `PeTTaEval s (superpose []) []`.
- `petta_eval_collapse_singleton`: `collapse` always produces a singleton.
- `spaceMatch_mono_addAtom`: adding a fact to the space only extends `spaceMatch` answers.

PeTTa is inherently non-deterministic: for a given expression p, multiple constructors of `PeTTaEval` may fire simultaneously (e.g., `.fvar x` can match both the `var` constructor and a `ruleApp` with an fvar-matching LHS).

6.3 Compiling PeTTa Spaces to LP

`Languages/MeTTa/PeTTa/LPSoundness.lean` (255 lines) defines the compilation:

```
def pettaSpaceToLangDef (s : PeTTaSpace) : LanguageDef where
  rewrites := s.rules   -- LP-safe rules are premise-free, hence
      root-only
  ...

def pettaSpaceToLPKB (s : PeTTaSpace) : KnowledgeBase mettailLPSig
    :=
  languageDefToLPKB (pettaSpaceToLangDef s)
```

Only `s.rules` is compiled; `s.facts` are handled by `spaceMatch`.

6.4 LP Soundness

Definition 11 (`pettaRuleSafe`)**.** *A rule r is* PeTTa LP-safe *if:* $\mathit{morkTranslatable}(r.left) = \mathit{true}$, $\mathit{morkTranslatable}(r.right) = \mathit{true}$, *and* $r.left.\mathit{isMatchCorrect} = \mathit{true}$. *This is exactly* `lpTranslatable r`.

Theorem 12 (`petta_ruleApp_lp_sound`)**.** *Let r be LP-safe with $r \in s.\mathit{rules}$, $r.\mathit{premises} = []$, $bs \in \mathit{matchPattern}(r.left, p)$, $\mathit{applyBindings}(bs, r.right) = q$. Then $\mathit{encodeReduces}(p, q) \in \mathrm{LHM}(\mathit{pettaSpaceToLPKB}(s))$.*

Proof. (1) `pettaRuleSafe_iff` unpacks the safety into the three components. (2) Theorem 7 (`matchPattern_correct`) applied to $bs \in$ `matchPattern`$(r.left, p)$ with `isMatchCorrect` gives `applyBindings`$(bs, r.left) = p$. (3) $r \in s.$`rules` lifts to $r \in$ (`pettaSpaceToLangDef`(s)).`rewrites` by `rfl`. (4) Theorem 10 (`lp_complete_topRule`) gives the conclusion. □

Corollary 13 (petta_safe_space_ruleApp_lp_sound)**.** *For an LP-safe space s (all rules satisfy* ***pettaRuleSafe****), every* ***ruleApp*** *derivation $r \in s.\mathtt{rules}$, bs, p, q is witnessed in* LHM(***pettaSpaceToLPKB***(s))*.*

Proof. Immediate from Theorem 12 using $hs(r, hr)$ for the safety condition. □

Theorem 14 (pettaSpaceToLPKB_addRule_mono)**.** *Adding a rule to a space preserves LP model membership: if $a \in$* LHM(***pettaSpaceToLPKB***(s))*, then $a \in$* LHM(***pettaSpaceToLPKB***$(s.\mathtt{addRule}(r))$)*.*

Proof. Key sub-lemma: for any interpretation I, $T_P(\mathtt{KB}(s), I) \subseteq T_P(\mathtt{KB}(s.\mathtt{addRule}(r)), I)$ (proven by noting that $s.\mathtt{rules} \subseteq r :: s.\mathtt{rules}$ and using `List.mem_cons_of_mem`). Then `leastHerbrandModel_least` (Theorem 4) with $I =$ LHM$(s.\mathtt{addRule}(r))$ being a pre-fixpoint of $T_P(KB(s))$ gives LHM$(s) \subseteq$ LHM$(s.\mathtt{addRule}(r))$. □

A sanity check completes the file: the empty space compiles to the empty KB, $\mathtt{pettaSpaceToLPKB}(\mathtt{empty}) = \{prog := [], db := \emptyset\}$.

6.5 Unification

The files `Unification.lean` (259 lines) and `UnificationMGU.lean` (229 lines) formalise unification for the LP kernel. The most general unifier (MGU) is computed and proved correct: `unify_mgu` states that any unifier factors through the MGU. These results are used in the SLD resolution soundness proof but are stated over the full LP signature (not restricted to `mettailLPSig`).

7 Certified Propositional ATP/Chainer Layer

`Logic/LP/PropositionalConnectionChainer.lean` (1,245 lines) provides a connection-tableau proof object language aligned with the PeTTa leanCoP-style canaries. The module contains:

- `TraceDerivable`/`ConnDerivable` (abstract derivability),
- executable replay checking (`replay`, `checkProof`),
- executable DFS search (`connProveDFS`),
- refinement and witness-level completeness theorems.

The main executable refinement theorems are:

Theorem 15 (connProveDFS_sound)**.** *If* ***connProveDFS*** *returns* ***some tr****, then the goal is abstractly connection-derivable.*

Theorem 16 (connProveDFS_witness_complete)**.** *If a goal is connection-derivable, then there exist finite fuel and a returned trace from* ***connProveDFS****.*

The fixture map section `PeTTaFixtureMap` provides theorem-level parity with PeTTa `ic_*` canaries.

PeTTa canary intent	**Lean theorem**	**Expected outcome**
UNSAT proof object accepted	`ic_unsat_checkProof_true`	`checkProof = true`
SAT singleton rejects DFS proof	`ic_sat_connProve_none`	`connProveDFS = none` (all fuel)
UNSAT DFS finds witness	`ic_unsat_connProve_has_proof`	$\exists tr$, `some` tr at fuel 2
Branching proof (root $\neg p$) accepted	`ic_branching_negp_checkProof_true`	`checkProof = true`
Branching proof (root $\neg q$) accepted	`ic_branching_negq_checkProof_true`	`checkProof = true`
Two distinct branching roots both valid	`ic_branching_two_distinct_root_proofs`	root inequality + both checks true
Branching DFS witness ($\neg p$ root)	`ic_branching_negp_connProve_has_proof`	$\exists tr$, `some` tr at fuel 3
Branching DFS witness ($\neg q$ root)	`ic_branching_negq_connProve_has_proof`	$\exists tr$, `some` tr at fuel 3

Table 1: Formal theorem map for PeTTa `ic_*` propositional canaries.

FO status is tracked in `Logic/LP/FirstOrderConnectionTrace.lean`. The proven FO endpoints now include replay-checker soundness (`replayFO_sound`), executable DFS soundness (`connProveFODFS_sound`), and witness-level DFS completeness (`connProveFODFS_witness_complete`).

This section therefore establishes theorem-prover-style executable refinement in Lean for the propositional fragment, with explicit FO soundness scaffolding already formalized and compiled.

8 File Statistics

File	Lines	Key theorems	Sorries
LP Kernel (Logic/LP/)			
Core.lean	243	LP signature, terms, clauses	0
Substitution.lean	206	grounding algebra	0
Semantics.lean	204	T_P, LHM, fixpoints	0
Matching.lean	333	LP term matching	0
SLD.lean	169	SLD soundness	0
SLDAll.lean	205	all-solutions SLD	0
SLDCompute.lean	485	semantic-lift completeness	0
SLDCompletenessKit.lean	261	canonical witness kits	0
SLDCompletenessCanaries.lean	178	completeness canaries	0
Unification.lean	259	unifyFuel soundness	0
UnificationMGU.lean	229	MGU factoring	0
UnificationComplete.lean	513	unifyFuel_exists_of_unifies	0
UnificationCompletenessCanaries.lean	122	occurs/non-occurs canaries	0
PropositionalConnectionChainer.lean	1,245	DFS soundness/completeness	0
PropositionalChainer.lean	265	propositional chainer	0
FirstOrderConnectionTrace.lean	1,440	FO connection trace	0
MeTTaILBridge.lean	847	commutation, LP completeness	0
FunctionFree.lean	109	function-free fragment	0
FunctionFreeEvaluation.lean	110	FF evaluation	0
FunctionFreePeTTa.lean	481	FF PeTTa integration	0
Provenance.lean	125	proof provenance	0
RangeRestriction.lean	197	range restriction	0
MMMeasure.lean	344	MM measure theory	0
OSLFBridge.lean	172	LP–OSLF bridge	0
PathMapBridge.lean	97	LP–PathMap bridge	0
WorldModelBridge.lean	92	LP–WM bridge	0
CertifyingDatalogBridge.lean	82	Datalog bridge	0
MeTTaIL Matching (OSLF/MeTTaIL/)			
Match.lean	201	algorithmic matcher	0
MatchSpec.lean	821	matchPattern_correct	0
Prolog Goal Language (Languages/Prolog/)			
Core.lean	220	PrologGoal inductive (14 ctors)	0
Eval.lean	443	Prolog evaluation relation	0
Prolog.lean	54	module re-export	0
FixtureCorpus.lean	2,379	ISO/Logtalk conformance fixtures	0
RuntimeErrorSpec.lean	62	ISO runtime-error boundary	0
PeTTa Specification (Languages/MeTTa/PeTTa/)			
Answers.lean	121	answer type	0
SpaceSemantics.lean	160	spaceMatch	0
Eval.lean	187	PeTTaEval	0
LPSoundness.lean	255	ruleApp LP soundness	0
Effects.lean	301	PeTTaCmd + state monotonicity	0
TypeSystem.lean	374	MeTTaType arrow typing	0
PrologBridge.lean	125	Prolog goal compilation	0
TranslateExpr.lean	463	compileExpr correctness	0
MeTTaEval.lean	398	4-arg MeTTaEval judgment	0
StdLib.lean	309	if-equal, car-atom, let	0
GroundedOracle.lean	297	grounded function oracles	0
TypedEval.lean	296	type-driven evaluation	0
MinimalInstructions.lean	388	minimal instruction set	0
OSLFInstance.lean	144	pettaOSLF, Galois, Rust export	0
GSLTVertex.lean	103	GSLT forward fiber	0
Total (49 files)	**17,114**		**0**

Table 2: File statistics for the certified PeTTa formalization. All files use only [propext, Classical.choice, Quot.sound].

9 Stateful Evaluation: Effects and Commands

`Languages/MeTTa/PeTTa/Effects.lean` (301 lines) extends the pure `PeTTaEval` relation with a stateful layer that models PeTTa's mutating atomspace operations.

9.1 Motivation

The pure `PeTTaEval` relation (Section 6) captures rule application, space queries, `superpose`, and `collapse`, but it treats the atomspace as a static read-only resource. The real PeTTa interpreter supports imperative mutations: `add-atom` and `remove-atom` modify `&self` in place; `get-atoms` retrieves all current facts; and `progn`/`prog1` sequence effects in a specific return-value discipline. Formalizing this requires a stateful layer.

9.2 Evaluation State

```
structure EvalState where
  space : PeTTaSpace
```

`EvalState` wraps a `PeTTaSpace` (the single `&self` atomspace). The namespace provides five operations:

```
def EvalState.empty     : EvalState
def EvalState.addAtom   (s : EvalState) (p : Pattern) : EvalState
def EvalState.removeAtom (s : EvalState) (p : Pattern) : EvalState
def EvalState.addRule   (s : EvalState) (r : RewriteRule) :
    EvalState
def EvalState.withSpace (s : EvalState) (sp : PeTTaSpace) :
    EvalState
```

All operations are purely functional: they return a new `EvalState`. The unit return value for `add-atom` and `remove-atom` is:

```
def unitAtom : Pattern := .apply "()" []
```

9.3 The `PeTTaCmd` Judgment

The stateful judgment `PeTTaCmd` s_0 `expr` s_1 `answers` means: starting from state s_0, evaluating `expr` transitions to state s_1 and produces nondeterministic answer set `answers`.

```
inductive PeTTaCmd : EvalState → Pattern → EvalState → Answers →
    Prop where
  | addAtomCmd (s : EvalState) (p : Pattern) :
      PeTTaCmd s
        (.apply "add-atom" [.apply "&self" [], p])
        (s.addAtom p)
        [unitAtom]
  | removeAtomCmd (s : EvalState) (p : Pattern) :
      PeTTaCmd s
```

```
      (.apply "remove-atom" [.apply "&self" [], p])
      (s.removeAtom p)
      [unitAtom]
| getAtomsCmd (s : EvalState) :
    PeTTaCmd s
      (.apply "get-atoms" [.apply "&self" []])
      s
      s.space.facts
| pureEval (s : EvalState) (p : Pattern) (answers : Answers)
    (h : PeTTaEval s.space p answers) :
    PeTTaCmd s p s answers
| prognCmd (s0 s1 s2 : EvalState) (e1 e2 : Pattern) (ans1 ans2 :
    Answers)
    (h1 : PeTTaCmd s0 e1 s1 ans1) (h2 : PeTTaCmd s1 e2 s2 ans2) :
    PeTTaCmd s0 (.apply "progn" [e1, e2]) s2 ans2
| prog1Cmd (s0 s1 s2 : EvalState) (e1 e2 : Pattern) (ans1 ans2 :
    Answers)
    (h1 : PeTTaCmd s0 e1 s1 ans1) (h2 : PeTTaCmd s1 e2 s2 ans2) :
    PeTTaCmd s0 (.apply "prog1" [e1, e2]) s2 ans1
```

The six constructors cover all effectful and pure evaluation forms. The `pureEval` constructor embeds any `PeTTaEval` derivation state-preservingly into the stateful layer. The `prognCmd` constructor sequences two commands, threading the intermediate state $\mathtt{s}_1$: the second command is evaluated in the output state of the first, and the final answers are those of the second. `prog1Cmd` is symmetric but returns the answers of the first command.

9.4 Correspondence with PeTTa Prolog

PeTTa Prolog predicate	`PeTTaCmd` **constructor**
`add_atom_to_space/2`	`addAtomCmd`
`remove_atom_from_space/2`	`removeAtomCmd`
`findall(A, get_atom(self,A), As)`	`getAtomsCmd`
pure `metta_call/2`	`pureEval`
`progn/2` (from `lib_metta4`)	`prognCmd`
`prog1/2` (from `lib_metta4`)	`prog1Cmd`

Table 3: Correspondence between PeTTa Prolog predicates and `PeTTaCmd` constructors.

9.5 Key Theorems

Lemma 17 (`pureEval_lifts`)**.** *Any `PeTTaEval` derivation lifts to a state-preserving `PeTTaCmd`: if `PeTTaEval s.space p ans`, then `PeTTaCmd s p s ans`.*

Lemma 18 (`addAtomCmd_mem_facts`)**.** *The added atom is a member of the output state's fact list: $p \in (\mathtt{s.addAtom}\ p).\mathtt{space.facts}$.*

Lemma 19 (`addAtomCmd_preserves_facts`)**.** *Existing facts survive an `addAtomCmd`: if $q \in \mathtt{s.space.facts}$ then $q \in (\mathtt{s.addAtom}\ p).\mathtt{space.facts}$.*

Lemma 20 (removeAtomCmd_subset_facts)**.** *`removeAtomCmd` only removes the targeted atom: any fact surviving in the output state was already present in the input state.*

Lemma 21 (pettaCmd_shape)**.** *Exhaustive case analysis: given any `PeTTaCmd s p s₁ ans`, exactly one of the following characterizes the expression form and state transition:*

1. $p =$ `(add-atom &self q)` *and* $s_1 =$ `s.addAtom` q,
2. $p =$ `(remove-atom &self q)` *and* $s_1 =$ `s.removeAtom` q,
3. $p =$ `(get-atoms &self)` *and* $s_1 = s$ *and* $ans =$ `s.space.facts`,
4. $s_1 = s$ *and* `PeTTaEval s.space` p ans,
5. $\exists e_1, e_2,\ p =$ `(progn` e_1 e_2`)`, *or*
6. $\exists e_1, e_2,\ p =$ `(prog1` e_1 e_2`)`.

Theorem 22 (example_addThenGet)**.** *Concrete derivation: evaluating `(progn (add-atom &self (foo)) (get-atoms &self))` from the empty state yields `[.apply "foo" []]`:*

```
theorem example_addThenGet :
    PeTTaCmd EvalState.empty
      (.apply "progn"
        [ .apply "add-atom" [.apply "&self" [], .apply "foo" []]
        , .apply "get-atoms" [.apply "&self" []] ])
      { space := { facts := [.apply "foo" []], rules := [] } }
      [.apply "foo" []] :=
  PeTTaCmd.prognCmd _ _ _ _ _ _ _
    (PeTTaCmd.addAtomCmd EvalState.empty (.apply "foo" []))
    (PeTTaCmd.getAtomsCmd _)
```

10 MeTTa Type System Fragment

`Languages/MeTTa/PeTTa/TypeSystem.lean` (374 lines) formalizes the static fragment of MeTTa's type system as an inductive judgment `MeTTaType s p t`, meaning "in atomspace `s`, pattern `p` has type `t`".

10.1 Special Type Atoms

MeTTa uses a handful of distinguished type-level atoms:

```
def undefinedType   : Pattern := .apply "%Undefined%" []  --
    supertype
def emptyType       : Pattern := .apply "%Empty%" []      -- bottom
def atomType        : Pattern := .apply "Atom" []         --
    nullary apps
def expressionType  : Pattern := .apply "Expression" []   -- all
    apps
```

```
def arrowType (A B : Pattern) : Pattern := .apply "↦" [A, B]
def typeAnnotationPat (p t : Pattern) : Pattern := .apply ":" [p, t
    ]
```

Type annotations are stored as facts in the atomspace: `(: p t)` $\in$ `s.facts` directly asserts that `p` has type `t`.

10.2 The Type Judgment

```
inductive MeTTaType (s : PeTTaSpace) : Pattern → Pattern → Prop
    where

  | typeAnnotation (p t : Pattern)
      (h : typeAnnotationPat p t in s.facts) :
      MeTTaType s p t

  | undefinedIsTop (p : Pattern) :
      MeTTaType s p undefinedType

  | arrowApp (c : String) (x argTy retTy : Pattern)
      (hf : MeTTaType s (.apply c []) (arrowType argTy retTy))
      (hx : MeTTaType s x argTy) :
      MeTTaType s (.apply c [x]) retTy

  | arrowAppUnknown (c : String) (x argTy retTy : Pattern)
      (hf : MeTTaType s (.apply c []) (arrowType argTy retTy)) :
      MeTTaType s (.apply c [x]) undefinedType

  | arrowMultiApp (c : String) (x argTy midTy retTy : Pattern)
      (hf : MeTTaType s (.apply c []) (arrowType argTy (arrowType
          midTy retTy)))
      (hx : MeTTaType s x argTy) :
      MeTTaType s (.apply c [x]) (arrowType midTy retTy)

  | symbolIsAtom (c : String) :
      MeTTaType s (.apply c []) atomType

  | appIsExpression (c : String) (args : List Pattern) (hne : args
      /= []) :
      MeTTaType s (.apply c args) expressionType

  | varIsUndefined (x : String) :
      MeTTaType s (.fvar x) undefinedType
```

The eight constructors cover the key cases from the MeTTa spec's `check_if_function_type_is_applicable` and `check_argument_type` predicates. Arrow constructors use `c : String` as the function head (since `Pattern.apply : String → List Pattern → Pattern` requires a string head); a bare symbol `f` is represented as `.apply c []`.

10.3 Design Decisions

Types as patterns. The type language is *the same* language as the term language; there is no separate sort. This mirrors MeTTa's philosophy of a

single, uniform atom space.

Non-determinism. `MeTTaType` is a relation, not a function: a pattern may have multiple types simultaneously (e.g., every pattern has `%Undefined%` via `undefinedIsTop`, and separately may have a specific annotated type).

Non-dependent fragment. We do not model type-level computation or the `Expression` type's special structural role in the full HE MeTTa evaluation path. Rich dependent-type style extensions are treated as post-spec extensions, not part of current HE alignment.

Function head restriction. Arrow rules require the function to be a bare symbol (`.apply c []`), not an arbitrary expression. The `arrowApp` constructor therefore carries `c : String` rather than a general pattern. This is the normal use case in MeTTa (function symbols are declared with `(: f (-> A B))` where `f` is a bare symbol).

10.4 Key Theorems

Theorem 23 (`all_spaces_well_typed`)**.** *Every `PeTTaSpace` is well-typed: for all `p`, `t`, if `(: p t)` ∈ `s.facts` then `MeTTaType s p t`.*

Proof. Directly by `MeTTaType.typeAnnotation`. □

Theorem 24 (`typeOf_mono_addAtom`)**.** *Type judgments are monotone: if `MeTTaType s p t`, then `MeTTaType (s.addAtom f) p t` for any new fact `f`.*

Proof. By induction on the `MeTTaType` derivation, with `PeTTaSpace.mem_facts_addAtom` for the annotation case. □

Theorem 25 (`every_pattern_has_undefined_type`)**.** *No pattern is type-less: `MeTTaType s p %Undefined%` for all `s` and `p`.*

Proof. Immediate by `MeTTaType.undefinedIsTop`. □

Theorem 26 (`arrowApp_implies_f_has_arrow_type`)**.** *If `(c x)` has a non-trivial type `t` (i.e., `t` ≠ `%Undefined%`, `t` ≠ `Expression`) that does not arise from a direct annotation `(: (c x) t)` ∈ `s.facts`, then the bare symbol `c` has some arrow type `(-> A B)` in `s`.*

Proof. By cases on the `MeTTaType` derivation for `(.apply c [x])`; the `typeAnnotation` case is ruled out by hypothesis, `undefinedIsTop` by $t \neq$ `%Undefined%`, `appIsExpression` by $t \neq$ `Expression`; the remaining arrow cases all carry an `hf` witness. □

10.5 Annotated Types Query

```
def PeTTaSpace.annotatedTypes (s : PeTTaSpace) (p : Pattern) : List
     Pattern :=
  s.facts.filterMap fun fact =>
    match fact with
    | .apply ":" [p', t] => if p' == p then some t else none
    | _                  => none
```

Theorem 27 (annotatedTypes_sound)**.** *Every type returned by* `annotatedTypes s p` *is derivable:* $t \in$ `annotatedTypes` $s\ p \Rightarrow$ `MeTTaType` $s\ p\ t$.

Proof. Via a private helper lemma `annotatedTypes_filterMap_aux` that performs case analysis on the `filterMap` body: only `.apply ":" [p', t']` facts where `p' = p` contribute, giving `(: p' t)` $\in$ `s.facts`, from which `typeAnnotation` applies. The `c` $\neq$ `":"` branch uses `split` + head-symbol extraction to derive a contradiction. □

10.6 Alignment with the MeTTa Specification

MeTTa spec predicate	`MeTTaType` constructor
`(: p t)` $\in$ `space`	`typeAnnotation`
`%Undefined%` as supertype	`undefinedIsTop`
`check_if_function_type_is_applicable`	`arrowApp`, `arrowMultiApp`
`check_argument_type` (unknown arg)	`arrowAppUnknown`
Bare symbols have type `Atom`	`symbolIsAtom`
Non-nullary applications have type `Expression`	`appIsExpression`
Unbound variables have type `%Undefined%`	`varIsUndefined`

The `arrowAppUnknown` constructor handles the MeTTa spec's fallback: when argument type checking fails (the argument type does not match or is unknown), the result type is `%Undefined%` rather than a type error.

11 Prolog Goal Language and Expression Compilation

PeTTa's `translator.pl` compiles MeTTa expressions into Prolog goals before evaluation. We formalize this pipeline in five files.

11.1 Prolog Goal Language

`Languages/Prolog/Core.lean` (220 lines) defines a Prolog-style goal language with 14 constructors covering the ISO Prolog control and meta-predicate

repertoire (re-exported via `Languages/Prolog/Prolog.lean` and referenced by `PrologBridge.lean`):

```
inductive PrologGoal where
  | succeed | fail | cut
  | conj      : PrologGoal → PrologGoal → PrologGoal
  | disj      : PrologGoal → PrologGoal → PrologGoal
  | ite       : PrologGoal → PrologGoal → PrologGoal
              → PrologGoal
  | once      : PrologGoal → PrologGoal
  | neg       : PrologGoal → PrologGoal
  | isVar     : Pattern → PrologGoal
  | unify     : Pattern → Pattern → PrologGoal
  | notUnify  : Pattern → Pattern → PrologGoal
  | findall   : String → PrologGoal → PrologGoal
  | spaceMatch : Pattern → Pattern → PrologGoal
  | reduceCall : List Pattern → PrologGoal
```

The constructors mirror ISO Prolog goal forms: `succeed`/`fail`/`cut` for control, `conj`/`disj`/`ite` for logical connectives, `once`/`neg` for determinism and negation-as-failure, `unify`/`notUnify` for unification predicates, `findall` for `findall/3`, `spaceMatch` for MeTTaIL space queries, and `reduceCall` for PeTTa's `reduce/2` predicate.

11.2 MeTTa Prolog Oracle

`PrologBridge.lean` wires the abstract `EvalOracle` to concrete PeTTa semantics:

```
def meTTaPrologOracle (s : PeTTaSpace) : EvalOracle where
  call args outs := match args with
    | [e] => PeTTaEval s e outs
    | _   => False
```

The semantic contract: `oracle.call [e] outs` ↔ `PeTTaEval s e outs`.

11.3 Expression Compilation (`compileExpr`)

`TranslateExpr.lean` (463 lines) defines the compilation pass and proves its correctness:

```
def compileExpr : Pattern → PrologGoal
  | .fvar x               => .reduceCall [.fvar x]
  | .apply c []           => .reduceCall [.apply c []]
  | .apply "superpose" [.collection .vec alts none] =>
      disjList (alts.map compileExpr)
  | .apply "collapse" [body] =>
      .findall "Out" (compileExpr body)
  | .apply "match" [.apply "&self" [], pat, tmpl] =>
      .spaceMatch pat tmpl
  | e                     => .reduceCall [e]
```

Theorem 28 (compileExpr_ruleApp_iff)**.** *For rule applications, compilation is sound and complete: a* `reduceCall` *Prolog evaluation of* `compileExpr (.apply c args)` *produces answer* `q` *if and only if* `PeTTaEval s (.apply c args) [q]` *holds via* `ruleApp`*.*

Theorem 29 (compileExpr_match_correct)**.** *For space queries,* `compileExpr (match &self pat tmpl)` *evaluates under the Prolog oracle to exactly the answers produced by* `PeTTaEval.spaceQuery`*.*

Theorem 30 (compileExpr_superpose_sound)**.** *For superpose, the compiled disjunction is sound: each branch of the Prolog disjunction corresponds to a member of the superpose alternatives list.*

The compilation covers `fvar`, ground atoms, `superpose`, `collapse`, `match`, `if`, `let`, `case`, and a conservative catch-all for unrecognised forms.

12 ISO Conformance Harness

A three-tier conformance suite validates `PrologEval` against the ISO Prolog standard, providing executable evidence that the formal semantics agrees with real Prolog implementations on a substantial fragment.

12.1 Lean-Aligned Fixtures

`FixtureCorpus.lean` (2,379 lines) contains 242 proven fixture theorems, each sourced from a specific Logtalk ISO test ID (e.g. `iso_true_0_01`, `iso_unify_2_17`) and proven by constructing the corresponding `PrologEval` derivation. A SWI-Prolog runner (`swi_fixture_runner.pl`) executes the same goals; a parity checker (`check_lean_swi_parity.py`) enforces that every `lean_aligned` case has both a Lean theorem and a passing SWI execution.

Current count: 183 `lean_aligned` cases covering 63 unique ISO IDs drawn from 9 upstream Logtalk test files (control: `true/0`, `fail/0`, conjunction, disjunction; predicates: `once/1`, `\+/1`, `=/2`, `\=/2`, `findall/3`).

12.2 Runtime-Error Boundary Probes

`RuntimeErrorSpec.lean` (62 lines) handles the boundary between the typed `PrologGoal` AST and ISO runtime errors. The AST intentionally excludes non-callable terms; four ISO cases that require runtime error detection are formalized as theorem-level boundary declarations mapping each ISO ID to its expected `RuntimeErrorClass`:

- `iso_not_1_06` ↦ `typeErrorCallable` (`\+ 3`)
- `iso_not_1_07` ↦ `instantiationError` (`\+ G` uninstantiated)
- `iso_findall_3_07` ↦ `instantiationError` (`findall(_, G, _)` uninstantiated)

- `iso_findall_3_08` ↦ `typeErrorCallable` (`findall(_, 4, _)`)

All four are proved by `rfl`.

12.3 Logtalk ISO-ID Coverage Gate

`report_logtalk_iso_coverage.py` extracts all `iso_*` identifiers from 9 upstream Logtalk test files (`https://github.com/LogtalkDotOrg/logtalk3`) and checks that every ID is represented by both a Lean theorem and a `lean_aligned` case. Hard threshold: 63/63 exact coverage for both theorem and case IDs.

12.4 Orchestration

The full suite is orchestrated by `scripts/prolog/run_conformance.sh`, which sequentially: (1) runs SWI fixture execution, (2) checks Lean↔SWI parity, (3) validates runtime-error boundary probes, and (4) reports Logtalk ISO-ID coverage with hard pass/fail thresholds.

12.5 PeTTa Runtime Unit Suite

The Lean conformance in Sections 12 and 11 proves properties of the *formal model*. A separate runtime suite tests the *concrete interpreter*: 5 files totalling 69 assertions, each exercising a distinct slice of PeTTa semantics (rewrite dispatch, constrained function-head matching, arithmetic and control flow, superposition/collapse non-determinism, and atomspace mutation/query). The files are distilled from canonical PeTTa examples (`functionhead3`, `spaces`, `matchnested`, `collapse`, `if2`–`if4`) and reduced to focused unit-scale witnesses.

A shell runner executes each file through the real PeTTa entrypoint and enforces two hard checks per file: zero failures and an exact pass count.

```
cd hyperon/PeTTa
./unit/run_petta_unit_69.sh
```

Together with the Lean theorem layer, this gives both proof-level and implementation-level validation of the PeTTa semantics. Appendix B maps each file to the spec features it exercises.

13 4-Argument MeTTa Evaluation

`Languages/MeTTa/PeTTa/MeTTaEval.lean` (398 lines) refines `PeTTaEval` with three orthogonal features: binding threading, error propagation, and type-driven pass-through.

13.1 The `MeTTaEval` Judgment

```
inductive MeTTaEval (s : PeTTaSpace)
    : Pattern → Pattern → Bindings → EvalResult → Prop where
  | symbolPassThrough (c : String) (ty : Pattern) (bs : Bindings) :
      MeTTaEval s (.apply c []) ty bs [(.apply c [], bs)]
  | varPassThrough (x : String) (ty : Pattern) (bs : Bindings) :
      MeTTaEval s (.fvar x) ty bs [(.fvar x, bs)]
  | errorPassThrough (atom msg : Pattern) (ty : Pattern) (bs :
      Bindings) :
      MeTTaEval s (.apply "Error" [atom, msg]) ty bs
        [(.apply "Error" [atom, msg], bs)]
  | ruleApp (r : RewriteRule) (mbs : Bindings) (p q : Pattern)
      (ty : Pattern) (bs : Bindings)
      (hr : r in s.rules) (hprem : r.premises = [])
      (hm : mbs in matchPattern r.left p)
      (hq : applyBindings mbs r.right = q) :
      MeTTaEval s p ty bs [(q, mbs ++ bs)]
  | spaceQuery ...
  | superpose ...
  | collapse ...
```

The type parameter `ty` flows through but does not affect reduction in the current fragment—it gates pass-through for special types (`%Undefined%`, `Atom`, `Expression`).

13.2 Embedding into `PeTTaEval`

Theorem 31 (`meTTaEval_ruleApp_to_pettaEval`)**.** *The* ***ruleApp*** *case of* ***MeTTaEval*** *projects faithfully to* ***PeTTaEval.ruleApp****.*

Similar projection theorems hold for `spaceQuery`, `superpose`, and `collapse`.

13.3 Standard Library (`StdLib.lean`, 309 lines)

`StdLib.lean` defines derived evaluation forms for PeTTa's standard operations: `if-equal` (conditional on pattern equality), `car-atom` (head extraction), `cdr-atom` (tail extraction), `let` and `let*` (sequential binding), `case` (pattern matching), and `sealed` (scope isolation). Each is defined as a specialized `MeTTaEval` step and proved sound against the base evaluation relation.

13.4 Grounded Oracles (`GroundedOracle.lean`, 297 lines)

`GroundedOracle.lean` formalizes the interface for grounded (external) functions in MeTTa: arithmetic, string operations, and type-checking predicates. A `GroundedFn` is a deterministic function `List Pattern → Option (List Pattern)`; the `groundedApp` evaluation constructor fires when the head symbol is a registered grounded function and the oracle succeeds.

13.5 Typed Evaluation (`TypedEval.lean`, 296 lines)

`TypedEval.lean` integrates `MeTTaType` (Section 10) with `MeTTaEval`: the `TypedEvalStep` judgment combines a type derivation with an evaluation step, ensuring that the input expression and its type are consistent. Key properties include type preservation under evaluation and type soundness for the pass-through cases.

14 OSLF Pipeline and GSLT Integration

`Languages/MeTTa/PeTTa/OSLFInstance.lean` (144 lines) and `Languages/MeTTa/PeTTa/GSLTVertex.lean` (103 lines) compose the PeTTa specification stack into the OSLF framework.

14.1 OSLF Type System

The key definition composes `pettaSpaceToLangDef` (from Section 6) with the generic OSLF type synthesis:

```
def pettaOSLF (s : PeTTaSpace) :=
  langOSLF (pettaSpaceToLangDef s) "Expr"

theorem pettaGalois (s : PeTTaSpace) :
    GaloisConnection (langDiamond (pettaSpaceToLangDef s))
                     (langBox (pettaSpaceToLangDef s)) :=
  langGalois (pettaSpaceToLangDef s)
```

The Galois connection $\Diamond \dashv \Box$ is obtained automatically from the generic OSLF infrastructure.

Theorem 32 (`pettaDiamond_spec`)**.** $\Diamond\varphi(p) \leftrightarrow \exists q,\ \mathit{langReduces}\ p\ q \wedge \varphi(q)$.

Theorem 33 (`pettaBox_spec`)**.** $\Box\varphi(p) \leftrightarrow \forall q,\ \mathit{langReduces}\ q\ p \to \varphi(q)$.

14.2 LP Soundness Bridge

Theorem 34 (`pettaOSLF_lp_sound`)**.** *For LP-safe* PeTTa *spaces, any* OSLF *diamond witness via* `ruleApp` *is backed by LP model membership.*

This composes `matchPattern_correct` (Section 4) with `lp_complete_topRule` (Section 5) through the OSLF pipeline.

14.3 Rust Export

```
def pettaRenderRust (s : PeTTaSpace) : String :=
  renderLanguage (pettaSpaceToLangDef s)

def pettaWriteRust (path : FilePath) (s : PeTTaSpace) : IO Unit :=
  writeLanguage path (pettaSpaceToLangDef s)
```

These produce `language! { ... }` macro text consumable by the `mettail-rust` runtime.

14.4 GSLT Forward Fiber

`GSLTVertex.lean` provides a unit-indexed GSLT forward fiber, following the governance pattern from the companion OSLF paper:

```
def pettaIdMorphism (s : PeTTaSpace) :
    ForwardMorphism (pettaSpaceToLangDef s)
                    (pettaSpaceToLangDef s) where
  mapTerm := id
  forward_sim _ q hred := <q, .single hred, rfl>

def pettaForwardFiber (s : PeTTaSpace) : ForwardFiber Unit where
  lang  := fun _ => pettaSpaceToLangDef s
  morph := fun _ => pettaIdMorphism s
```

This places PeTTa as a degenerate (constant) vertex in the GSLT hypercube, enabling categorical composition with other language fibers (PathMap, Governance, Temporal).

15 Related Work

LP semantics. The Herbrand model semantics for LP was established by van Emden and Kowalski [**?**], who proved the equivalence of operational (SLD resolution) and declarative (least model) semantics for Horn clause programs. Lloyd's monograph [**?**] remains the standard reference. Our formalization follows Chapter 2 of Lloyd directly, using `OrderHom.lfp` from Mathlib to realise Tarski's theorem on the set lattice.

Lean 4 LP/logic formalization. The Mettapedia project contains a separate Datalog formalization (formerly in `Mettapedia/Logic/Datalog/`, now consolidated into `Logic/LP/`), which covered the Datalog fragment without function symbols. The present LP kernel generalises this to function symbols, enabling the pattern encoding. We are not aware of other full-stack LP-to-pattern-matching soundness formalizations in Lean 4.

MeTTa and Hyperon. MeTTa and the Hyperon architecture are described in Goertzel et al. [**?**]. The PLN reasoning layer is formalized in other Mettapedia papers (see `Mettapedia/PLN/Core/PLNCore.lean` et al.). The OSLF framework and its modal type system are formalized separately in the companion paper [**?**].

Certified pattern matching. Pattern matching correctness for a locally-nameless λ-calculus was studied by Aydemir et al. [**?**]. Our `isMatchCorrect`

fragment corresponds to the "linear" fragment where each pattern variable occurs exactly once, plus the exclusion of collection patterns whose bag-matching non-determinism prevents uniqueness of the reconstructed term.

Prolog formalization. Formalizations of SLD resolution correctness are a classical topic; our treatment follows Lloyd's presentation [**?**] mechanized in Lean 4's type theory.

Connection tableaux and leanCoP lineage. Connection-style calculi go back to Bibel's matrix/connection method tradition [**?**]. leanCoP operationalized this line with a compact Prolog implementation and practical connection-tableau search control [**?**]. Our propositional chainer layer is not a full reproof of leanCoP, but it follows the same proof-object shape (reduce/extend traces), and, crucially, adds Lean-certified replay soundness and executable DFS refinement theorems.

16 Conclusion and Future Work

We have presented a Lean 4 formalization of the PeTTa language comprising 17,114 lines across 49 files with zero sorries and zero custom axioms. The formalization establishes a certified end-to-end pipeline: from LP foundations (Herbrand model, SLD resolution, unification completeness) through MeTTaIL pattern matching correctness and LP soundness for PeTTa rule application, to a Prolog-style goal language with expression compilation, a full 4-argument evaluation judgment with grounded oracles and standard library forms, and finally an OSLF type system with automatic $\Diamond \dashv \Box$ Galois connection and Rust code export.

Future work.

- **Contextual LP compilation and converse direction.** The current bridge proves least-model membership for a supplied authored-rule application. A full reduction-level bridge must compile authored `Premise.congruence` rules rather than collection representation, and a converse from LP membership requires range restriction plus a constructive lifting theorem from LP witnesses back into MeTTaIL reductions. Range restriction (`RangeRestriction.lean`) is partially formalized.
- **First-order connection search refinement.** The propositional connection-tableau layer is fully certified; extending executable refinement and witness-level completeness to the full first-order trace/substitution layer (`FirstOrderConnectionTrace.lean`) is ongoing work.
- **HE type-system completion.** The current type-system fragment covers annotation lookup, arrow application, and pass-through

types. Completing full HE alignment requires formalizing the remaining `check_if_function_type_is_applicable` / `check_argument_type` branches (including `BadArgType`/`BadType` paths, non-symbol operator handling, and `Expression` return-type behavior).

- **Multi-space evaluation.** The current formalization uses a single `&self` atomspace. Extending to multi-space evaluation (`new-space`, `bind!`) would capture the full MeTTa import and module system.

A Theorem Index

Layer	Canonical theorem	Status
LP Kernel		
Unification (FO, semantic ⇒ executable)	`unifyFuel_exists_of_unifies`	proved
Unification (function-free)	`unifyFuel_exists_of_unifies_functionFree`	proved
SLD query completeness from semantic lift	`sldQuery_complete_of_semantic_lift`	proved
SLD query complete+sound bundle	`sldQuery_complete_sound_bundle_of_semantic_lift`	proved
ATP/Chainer		
Propositional connection DFS soundness	`connProveDFS_sound`	proved
Propositional connection DFS witness completeness	`connProveDFS_witness_complete`	proved
FO connection DFS soundness	`connProveFODFS_sound`	proved
FO connection DFS witness completeness	`connProveFODFS_witness_complete`	proved
MeTTaIL/PeTTa Bridge		
Pattern matching correctness	`matchPattern_correct`	proved
Authored rule-application LP encoding	`ruleApplication_mem_leastHerbrandModel`	proved
PeTTa ruleApp LP soundness	`petta_ruleApp_lp_sound`	proved
Prolog Compilation		
compileExpr ruleApp correctness	`compileExpr_reduceCall_iff`	proved
compileExpr match correctness	`compileExpr_match_correct`	proved
compileExpr superpose soundness	`compileExpr_superpose_sound`	proved
OSLF Pipeline		
PeTTa Galois connection	`pettaGalois`	proved
OSLF LP soundness bridge	`pettaOSLF_lp_sound`	proved
GSLT forward fiber	`pettaForwardFiber`	proved

Table 4: Compact theorem index for the certified PeTTa formalization.

B Runtime Unit Coverage Matrix

File	Tests	Features exercised
`01_core_rewrite.metta`	15	Deterministic rewrite, symbol/term pass-through, basic `superpose/collapse`
`02_function_head.metta`	8	Constrained function-head matching (`in`), guarded arithmetic, membership filtering
`03_arith_control.metta`	17	Grounded arithmetic/comparison, `if`, `let`, `let*`
`04_superpose_nondet.metta`	10	Nested non-determinism, `superpose/collapse`, sorted answer normalization
`05_atomspace_match.metta`	19	`add-atom/remove-atom/match` over `&self` and `&wuspace`, join-like queries

Table 5: PeTTa runtime unit suite: 5 files, 69 tests.